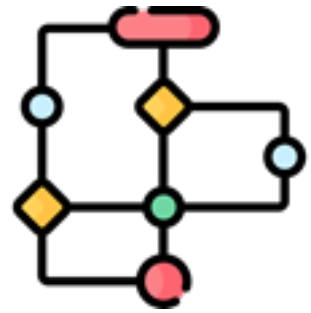
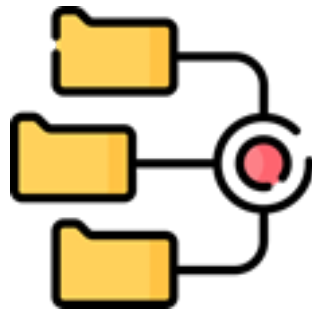




Структуры данных и алгоритмы



Содержание

1	Оценки алгоритмов	4
1.1	Оценка скорости алгоритмов	5
1.2	Оценка памяти необходимой алгоритмам	6
1.3	Стабильность алгоритмов	6
1.4	Адаптивность алгоритмов	6
1.5	Детерминированность алгоритмов	6
2	Структуры данных	6
2.1	Классификация структур данных	7
2.2	Линейные структуры данных	7
2.2.1	Массив (Array)	7
2.2.2	Стек (Stack)	8
2.2.3	Очередь (Queue)	8
2.2.4	Дек (Deque)	8
2.2.5	Очередь с приоритетом (Priority queue)	9
2.2.6	Односвязный список (Singly linked list)	9
2.2.7	Двусвязный список (Doubly linked list)	10
2.2.8	Кольцевой связный список (Circular linked list)	10
2.2.9	Строка (String)	10
2.3	Нелинейные структуры данных	11
2.3.1	Бинарное дерево (Binary tree)	11
2.3.2	Двоичное дерево поиска (Binary search tree — BST)	12
2.3.3	АВЛ-дерево (AVL tree)	12
2.3.4	Красно-чёрное дерево (Red-black tree)	14
2.3.5	Дерево отрезков (Segment tree)	14
2.3.6	В-дерево (B-tree)	14
2.3.7	В+-дерево (B+-tree)	15
2.3.8	Куча (Heap)	15
2.3.9	Двоичная куча (Binary heap)	15
2.3.10	Префиксное дерево (Trie)	16
2.3.11	Декартово дерево (Treap/Cartesian tree)	16
2.3.12	Splay-дерево (Splay tree)	16
2.3.13	Дерево квадрантов (Quadtree)	17
2.3.14	Дерево восьмиричных разбиений (Octree)	17
2.3.15	Арифметическое дерево выражения (Expression tree)	17
2.3.16	Неориентированный граф (Undirected graph)	17
2.3.17	Ориентированный граф (Directed graph/Digraph)	18
2.3.18	Взвешенный граф (Weighted graph)	18
2.3.19	Граф с метками (Labeled graph)	18
2.3.20	Лес (Forest)	19
2.3.21	Связный граф (Connected graph)	19
2.3.22	Полный граф (Complete graph)	19
2.3.23	Двудольный граф (Bipartite graph)	20
2.3.24	Ациклический ориентированный граф (Directed acyclic graph — DAG)	20
2.3.25	Матрица (Matrix)	20
2.3.26	Тензор (Tensor)	20

3	Алгоритмы	21
3.1	Алгоритмы сортировки	21
3.1.1	Сортировка пузырьком (Bubble sort)	21
3.1.2	Сортировка выбором (Selection sort)	22
3.1.3	Сортировка вставками (Insertion sort)	22
3.1.4	Сортировка слиянием (Merge sort)	22
3.1.5	Быстрая сортировка (Quick sort)	22
3.1.6	Пирамидальная сортировка (Heap sort)	23
3.1.7	Сортировка Шелла (Shell sort)	23
3.1.8	Сортировка деревом (Tree sort)	23
3.1.9	Сортировка подсчётом (Counting sort)	24
3.1.10	Поразрядная сортировка (Radix sort)	24
3.1.11	Блочная/Карманная сортировка (Bucket sort)	24
3.1.12	Тимо-сортировка (Tim sort)	24
3.1.13	Интроспективная сортировка (Introspective sort)	25
3.1.14	Гномья сортировка (Gnome sort)	25
3.1.15	Сортировка расчёской (Comb sort)	25
3.1.16	Глупая сортировка (Bogo sort)	26
3.2	Алгоритмы поиска	26
3.2.1	Линейный поиск (Linear search)	26
3.2.2	Двоичный поиск (Binary search)	26
3.2.3	Поиск прыжками (Jump search)	26
3.2.4	Интерполяционный поиск (Interpolation search)	27
3.2.5	Экспоненциальный поиск (Exponential search)	27
3.2.6	Поиск хеш-таблицой (Hash table search)	27
3.2.7	Поиск в бинарном деревом (Binary tree search)	27
3.2.8	Поиск делением на три части (Ternary search)	28
3.3	Алгоритмы на графах	28
3.3.1	Поиск в ширину (Breadth-first search — BFS)	28
3.3.2	Поиск в глубину (Depth-first search — DFS)	28
3.3.3	Алгоритм Дейкстры (Dijkstra algorithm)	28
3.3.4	Алгоритм Беллмана-Форда (Bellman-Ford algorithm)	29
3.3.5	Алгоритм Флойда-Уоршелла (Floyd-Warshall algorithm)	29
3.3.6	Алгоритм Прима (Prim algorithm)	29
3.3.7	Алгоритм Краскала (Kruskal algorithm)	29
3.3.8	Алгоритм Форда-Фалкерсона (Ford-Fulkerson algorithm)	30
3.3.9	Алгоритм Эдмондса-Карпа (Edmonds-Karp algorithm)	30
3.3.10	Алгоритм Диница (Dinic algorithm)	30
3.3.11	Поиск компонент связности (Connected components)	30
3.3.12	Поиск мостов и точек сочленения (Bridges & Articulation points)	30
3.3.13	Алгоритм Косараяю (Kosaraju algorithm)	31
3.3.14	Алгоритм Тарьяна (Tarjan algorithm)	31
3.3.15	Топологическая сортировка (Topological sort)	31
3.3.16	Поиск эйлерова пути (Eulerian path)	31
3.3.17	Поиск гамильтонова пути (Hamiltonian path)	32
3.3.18	Раскраска графов (Graph coloring)	32
3.3.19	Паросочетания в графах (Graph matching)	32
3.3.20	Поиск A* (A* search)	32

3.3.21	Поиск по первому лучшему (Best-first search)	32
3.3.22	Жадный поиск (Greedy search)	33
3.4	Алгоритмы на строках	33
3.4.1	Наивный алгоритм (Naive algorithm)	33
3.4.2	Алгоритм Кнута-Морриса-Пратта (Knuth-Morris-Pratt algorithm)	33
3.4.3	Алгоритм Рабина-Карпа (Rabin-Karp algorithm)	33
3.4.4	Алгоритм Бойера-Мура (Boyer-Moore algorithm)	34
3.4.5	Алгоритм Ахо-Корасика (Aho-Corasick algorithm)	34
3.4.6	Расстояние Левенштейна (Levenshtein distance)	34
3.4.7	Расстояние Хэмминга (Hamming distance)	34
3.4.8	Наибольшая общая подпоследовательность (Longest common subsequence — LCS)	35
3.4.9	Наибольшая общая подстрока (Longest common substring)	35
3.4.10	Алгоритм Вагнера-Фишера (Wagner-Fisher algorithm)	35
4	Алгоритмы преобразования данных	35
4.1	Алгоритмы сжатия данных	35
4.1.1	Алгоритм Хаффмана (Huffman coding)	36
4.1.2	Алгоритм Лемпеля-Зива (LZ77, LZ78, LZW)	36
4.1.3	Арифметическое кодирование (Arithmetic coding)	36
4.1.4	Кодирование длин серий (Run-length encoding - RLE)	36
4.1.5	BWT-преобразование (Burrows-Wheeler transform)	37
4.1.6	Deflate	37
4.1.7	Brotli	37
4.1.8	LZ4	37
4.1.9	Zstandard	38
4.2	Криптографические алгоритмы	38
4.2.1	AES (Advanced Encryption Standard)	38
4.2.2	DES/3DES (Data Encryption Standard)	38
4.2.3	Blowfish/Twofish	38
4.2.4	ChaCha20	39
4.2.5	RC4	39
4.2.6	RSA (Rivest-Shamir-Adleman)	39
4.2.7	ECC (Elliptic Curve Cryptography)	39
4.2.8	ElGamal	40
4.2.9	DSA (Digital Signature Algorithm)	40
4.2.10	SHA-256/SHA-3 (Secure Hash Algorithm)	40
4.2.11	MD5	40
4.2.12	Bcrypt/Argon2	41
4.2.13	Diffie-Hellman	41
4.2.14	PKI (Public Key Infrastructure)	41
4.2.15	Цифровые подписи (Digital signatures)	41

1 Оценки алгоритмов

Оценка алгоритмов — это процесс анализа и сравнения алгоритмов для определения их эффективности в зависимости от размера входных данных n .

Для оценки алгоритмов используются разные нотации. Эти нотации нужны для оценки работы алгоритмов в худшем или лучшем случае или как себя введет алгоритм в среднем. Для этого нам нужно описать алгоритм $f(n)$ какой-то более простой функцией $g(n)$.

Виды нотаций:

- **Верхняя граница** (худший случай или нотация О-большое): $f(n) = O(g(n))$, если существуют положительные константы c и n_0 такие, что $0 \leq f(n) \leq c \cdot g(n)$ для всех $n \geq n_0$. Это означает, что алгоритм работает не медленнее, чем указанная функция.
- **Нижняя граница** (лучший случай или нотация Омега-большое): $f(n) = \Omega(g(n))$, если существуют положительные константы c и n_0 такие, что $0 \leq c \cdot g(n) \leq f(n)$ для всех $n \geq n_0$. Это означает, что алгоритм работает не быстрее, чем указанная функция.
- **Точная граница** (средний случай или нотация Тета): $f(n) = \Theta(g(n))$, если существуют положительные константы c_1 , c_2 и n_0 такие, что $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ для всех $n \geq n_0$. Это означает, что алгоритм работает примерно так же, как указанная функция.
- **Строгая верхняя граница** (нотация о-малое): $f(n) = o(g(n))$, если для любой положительной константы $c > 0$ существует константа $n_0 > 0$ такие, что $0 \leq f(n) < c \cdot g(n)$ для всех $n \geq n_0$. Это означает, что алгоритм работает существенно быстрее, чем указанная функция.
- **Строгая нижняя граница** (нотация омега-малое): $f(n) = \omega(g(n))$, если для любой положительной константы $c > 0$ существует константа $n_0 > 0$ такие, что $0 \leq c \cdot g(n) < f(n)$ для всех $n \geq n_0$. Это означает, что алгоритм работает существенно быстрее, чем указанная функция.

Строгие оценки используются для более тонких асимптотических сравнений. Например, наш алгоритм можно описать функцией $f(n) = 2 \cdot n$ и мы пытаемся оценить его функцией $g(n) = n$. Тогда проверим следующие равенства:

- $2 \cdot n = O(n)$ — верно, так как мы можем подобрать константу c такую, что будет выполняться неравенство.
- $2 \cdot n = o(n)$ — неверно, так как неравенство должно выполняться для любой положительной константы c .
- $2 \cdot n = \Omega(n)$ — верно, так как мы можем подобрать константу c такую, что будет выполняться неравенство.
- $2 \cdot n = \omega(n)$ — неверно, так как неравенство должно выполняться для любой положительной константы c .

Теперь рассмотрим какими функциями $g(n)$ мы можем оценивать алгоритм:

- $g(n) = \text{const}$ — константная сложность, алгоритм не зависит от размера входных данных и часто константу берут равной единице ($\text{const} = 1$).
- $g(n) = \log(n)$ — логарифмическая сложность.
- $g(n) = n$ — линейная сложность.
- $g(n) = n \cdot \log(n)$ — линейно-логарифмическая сложность (квазилинейная сложность).
- $g(n) = n^{\text{const}}$ — степенная сложность (полиномиальная сложность), частные случаи n^2 — квадратичная сложность и n^3 — кубическая сложность.
- $g(n) = \text{const}^n$ — показательная сложность (экспоненциальная сложность) причем $c > 1$, частные случаи 2^n и 3^n .
- $g(n) = n!$ — факториальная сложность.

Вот график для сравнения функций оценок сложности:

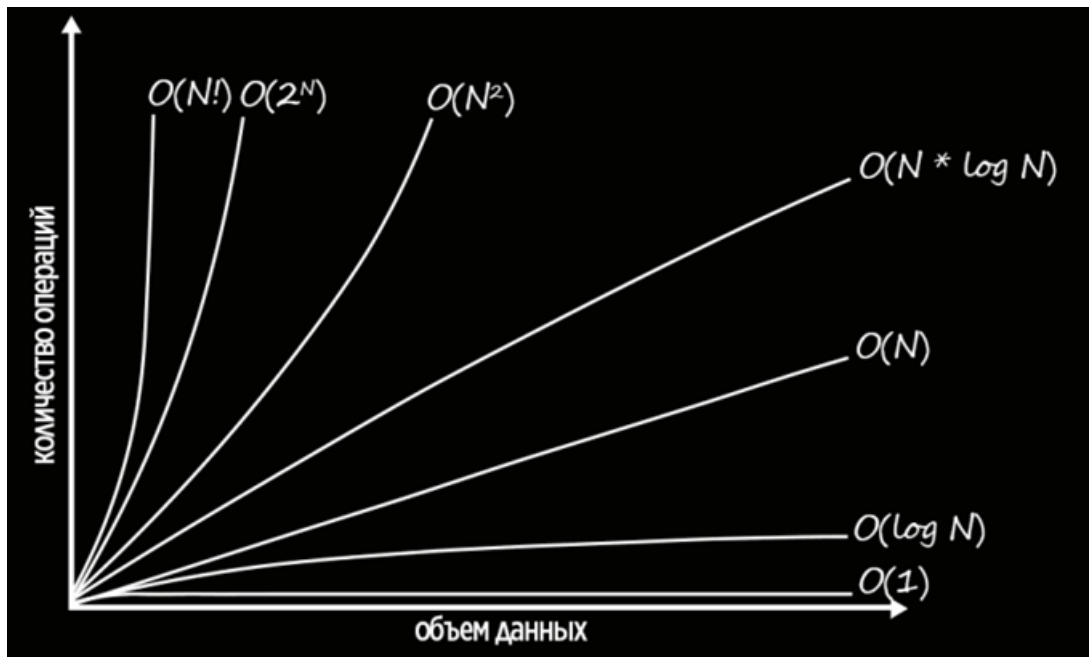


Рис. 1: Сравнение функций оценок сложности

Таковыми оценками мы можем оценить скорость работы алгоритма (временная сложность алгоритма) или память необходимую алгоритму (пространственная сложность). Также подобные оценки используют не только для оценок алгоритмов, но во многих других сферах.

1.1 Оценка скорости алгоритмов

Оценка скорости алгоритма или временная сложность, — это мера того, как время выполнения алгоритма увеличивается с ростом размера входных данных n . С помощью этой оценки мы можем выбрать алгоритм, когда нам важна скорость работы.

1.2 Оценка памяти необходимой алгоритмам

Оценка памяти необходимой алгоритму или пространственная сложность — это оценка объема памяти, которую алгоритм использует для своей работы в зависимости от размера входных данных n . Можем использовать данные оценки, если нам критически важно отслеживать память, используемую алгоритмом и наши ресурсы ограничены.

Также данные оценки могут показать использует ли алгоритм дополнительную память. Например, некоторые алгоритмы могут создавать копию входных данных и модернизировать именно эту копию и параллельно записывать правильно модернизированные данные во входные. Алгоритмы, которые работают без создания копий входных данных и меняют их непосредственно в оригинальном наборе, называют алгоритмами, работающими на месте.

1.3 Стабильность алгоритмов

Стабильность — это свойство алгоритмов сортировки, которое гарантирует, что равные элементы сохранят свой относительный порядок в отсортированном выводе. Это иногда бывает очень важно, когда входные данные имеют несколько параметров по которым нужно сортировать. Если мы, используя стабильный алгоритм сортировки, отсортируем сначала по первому параметру, а потом по второму, то данные будут отсортированы по второму параметру, но их отсортированный порядок по первому параметру также сохраниться.

1.4 Адаптивность алгоритмов

Адаптивность — это свойство алгоритма, позволяющее ему эффективно работать на частично отсортированных данных. Адаптивные алгоритмы экономят время и ресурсы, если входные данные уже имеют некоторую степень упорядоченности и на почти отсортированных алгоритмах их оценка должна стремиться к $g(n) = n$.

1.5 Детерминированность алгоритмов

Детерминированность — это свойство алгоритма, означающее, что для одного и того же набора входных данных он всегда выдаст идентичный результат и пройдет одни и те же шаги. Просто иногда алгоритм может выбирать в своей работе случайные значения какого-то элемента необходимого для работы и от этого выбора будет зависеть скорость работы алгоритма. Детерминированные алгоритмы предсказуемы и их легко отлаживать, однако недетерминированные алгоритмы могут быть полезны для.

2 Структуры данных

Структуры данных — это специализированные способы организации, хранения и управления данными, которые обеспечивают эффективный доступ и модификацию. Если алгоритм — это набор инструкций для решения задачи, то структура данных — это инструмент, который алгоритм использует для работы с информацией.

Правильный выбор структуры данных является фундаментальным для создания эффективных программ. Он напрямую влияет на производительность алгоритмов, которые с этими данными работают.

Ключевые аспекты:

- **Операции:** каждая структура данных поддерживает определенный набор операций (добавление, удаление, поиск, доступ), которые могут выполняться с разной эффективностью.
- **Эффективность:** выбор между одной структурой данных и другой часто представляет собой компромисс между скоростью выполнения различных операций и объемом используемой памяти.
- **Абстракция:** структуры данных предоставляют абстрактный интерфейс для работы с данными, скрывая внутренние детали их реализации и берут низкоуровневое хранение в памяти на себя.

2.1 Классификация структур данных

Основным разделением структур данных является разделение по организации элементов. Мы можем разделить структуры на линейные (элементы образуют последовательность, где каждый элемент связан с предыдущим и/или следующим) и нелинейные (элементы не образуют линейной последовательности и в таких структурах один элемент может быть связан с несколькими другими). Данная классификация является наиболее часто встречаемой и считается фундаментальной поскольку показывает, как данные в структуре связаны между собой.

Также можно классифицировать структуры данных по типу памяти на статические (размер структуры фиксирован и определяется на этапе компиляции) и динамические (размер структуры может изменяться во время выполнения программы). Но чаще данные классифицируют именно по организации элементов, а организацию памяти в структуре указывают как техническую деталь реализации структуры.

В этой главе мы опишем структуры данных: реализация структур, доступные операции над данными в структуре, сложностью выполнений каждой из операций.

2.2 Линейные структуры данных

Линейные структуры данных — это структуры, в которых элементы расположены последовательно, один за другим, образуя цепочку. Каждый элемент (кроме первого и последнего) имеет ровно одного предшественника («соседа слева») и одного преемника («соседа справа»).

2.2.1 Массив (Array)

Массивы бывают статические, когда их размер фиксированный и не меняется в ходе выполнения программы, и динамическими, когда память, выделенная под массив, может менять свой размер в ходе выполнения программы. Но главной особенностью массива является то, что он хранится в непрерывном блоке памяти, то есть значения всех элементов массива хранятся друг за другом в памяти.

Операция	Лучший случай	Средний случай	Худший случай
Доступ по индексу	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$
Вставка в начало	$O(n)$	$O(n)$	$O(n)$
Вставка в конец	$O(1)$	$O(1)$	$O(n)$
Вставка в середину	$O(n)$	$O(n)$	$O(n)$
Удаление из начала	$O(n)$	$O(n)$	$O(n)$
Удаление из конца	$O(1)$	$O(1)$	$O(1)$
Удаление из середины	$O(n)$	$O(n)$	$O(n)$

Таблица 1: Сложность операций для массива

2.2.2 Стек (Stack)

Стек — это структура данных, реализовываемая на принципе LIFO (Last-In, First-Out), то есть элемент, пришедший последним будет доставаться первым из стека.

Операция	Лучший случай	Средний случай	Худший случай
Добавление элемента	$O(1)$	$O(1)$	$O(n)$
Извлечение элемента	$O(1)$	$O(1)$	$O(1)$
Просмотр элемента без извлечения	$O(1)$	$O(1)$	$O(1)$
Проверка на пустоту	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 2: Сложность операций для стека

2.2.3 Очередь (Queue)

Очередь — это структура данных, реализовываемая на принципе FIFO (First-In, First-Out), то есть элемент, пришедший первым будет доставаться первым из стека.

Операция	Лучший случай	Средний случай	Худший случай
Добавление элемента	$O(1)$	$O(1)$	$O(n)$
Извлечение элемента	$O(1)$	$O(1)$	$O(n)$
Просмотр элемента без извлечения	$O(1)$	$O(1)$	$O(1)$
Проверка на пустоту	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 3: Сложность операций для очереди

2.2.4 Дек (Deque)

Дек — это улучшенная версия очереди, которая позволяет добавлять и удалять элементы как с начала, так и с конца структуры. Сочетает возможности стека и очереди в одной структуре.

Операция	Лучший случай	Средний случай	Худший случай
Добавление в начало	$O(1)$	$O(1)$	$O(n)$
Добавление в конец	$O(1)$	$O(1)$	$O(n)$
Извлечение из начала	$O(1)$	$O(1)$	$O(1)$
Извлечение из конца	$O(1)$	$O(1)$	$O(1)$
Просмотр начала без извлечения	$O(1)$	$O(1)$	$O(1)$
Просмотр конца без извлечения	$O(1)$	$O(1)$	$O(1)$
Проверка на пустоту	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 4: Сложность операций для дека

2.2.5 Очередь с приоритетом (Priority queue)

Очередь с приоритетом — это структура данных, где каждый элемент имеет приоритет. Элементы извлекаются не в порядке добавления (как в обычной очереди), а в порядке приоритета — сначала элементы с наивысшим приоритетом. Это делает её идеальной для задач планирования и обработки событий.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Извлечение элемента с высшим приоритетом	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Извлечение элемента с низшим приоритетом	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Просмотр элемента с высшим приоритетом без извлечения	$O(1)$	$O(1)$	$O(1)$
Просмотр элемента с низшим приоритетом без извлечения	$O(1)$	$O(1)$	$O(1)$
Изменение приоритета элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Объединение двух очередей с приоритетом размерами n и m	$O(1)$	$O(m \cdot \log(n))$	$O(m \cdot \log(n))$
Проверка на пустоту	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 5: Сложность операций для очереди с приоритетом

2.2.6 Односвязный список (Singly linked list)

Односвязный список — это динамическая структура данных, состоящая из узлов, где каждый узел содержит данные и ссылку на следующий узел. Это позволяет эффективно вставлять и удалять элементы в любом месте списка, но доступ к произвольному элементу требует последовательного прохода.

Операция	Лучший случай	Средний случай	Худший случай
Вставка в начало	$O(1)$	$O(1)$	$O(1)$
Вставка в конец	$O(1)$	$O(n)$	$O(n)$
Удаление из начала	$O(1)$	$O(1)$	$O(1)$
Удаление из конца	$O(1)$	$O(1)$	$O(1)$
Доступ по индексу	$O(1)$	$O(n)$	$O(n)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 6: Сложность операций для односвязного списка

2.2.7 Двусвязный список (Doubly linked list)

Двусвязный список — это улучшенная версия односвязного списка, где каждый узел содержит две ссылки: на следующий и на предыдущий узел. Это позволяет обходить список в обоих направлениях и упрощает некоторые операции, но требует больше памяти для хранения дополнительных указателей.

Операция	Лучший случай	Средний случай	Худший случай
Вставка в начало	$O(1)$	$O(1)$	$O(1)$
Вставка в конец	$O(1)$	$O(1)$	$O(1)$
Удаление из начала	$O(1)$	$O(1)$	$O(1)$
Удаление из конца	$O(1)$	$O(1)$	$O(1)$
Доступ по индексу	$O(1)$	$O(n)$	$O(n)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 7: Сложность операций для двусвязного списка

2.2.8 Кольцевой связный список (Circular linked list)

Кольцевой связный список — это модификация связного списка, в которой последний узел ссылается на первый, образуя замкнутый круг. Может быть как односвязным, так и двусвязным. Особенно полезен для циклических процессов и данных, где нужно непрерывно перебирать элементы.

Операция	Лучший случай	Средний случай	Худший случай
Вставка узла	$O(1)$	$O(1)$	$O(1)$
Удаление узла	$O(1)$	$O(1)$	$O(1)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$

Таблица 8: Сложность операций для кольцевого связного списка

2.2.9 Строка (String)

Строка — это последовательность символов, представляющая текст. Хотя логически строка является линейной структурой, физически может реализовываться различными способами. Это одна из самых часто используемых структур данных в программировании.

Операция	Лучший случай	Средний случай	Худший случай
Вставка символа в начало	$O(n)$	$O(n)$	$O(n)$
Вставка символа в конец	$O(1)$	$O(1)$	$O(n)$
Вставка символа в середину	$O(n)$	$O(n)$	$O(n)$
Удаление символа из начала	$O(n)$	$O(n)$	$O(n)$
Удаление символа из конца	$O(1)$	$O(1)$	$O(1)$
Удаление символа из середины	$O(n)$	$O(n)$	$O(n)$
Подсчет длины строки	$O(1)$	$O(n)$	$O(n)$
Сравнение строк	$O(1)$	$O(n)$	$O(n)$
Поиск в строке подстроки длины k	$O(1)$	$O(n)$	$O(n \cdot k)$
Доступ к символу по индексу	$O(1)$	$O(1)$	$O(1)$
Разделение строки	$O(1)$	$O(n)$	$O(n)$

Таблица 9: Сложность операций для строки

2.3 Нелинейные структуры данных

Нелинейные структуры данных — это структуры, в которых каждый элемент может быть связан с одним или несколькими другими элементами, не образуя линейной последовательности. Связи между элементами носят более сложный, иерархический или сетевой характер.

В свою очередь нелинейные структуры можно также разделить на некоторые типы: древовидные структуры, графы и многомерные структуры.

2.3.1 Бинарное дерево (Binary tree)

Бинарное дерево — это иерархическая структура данных, в которой каждый узел имеет не более двух потомков (левый и правый). Это базовая структура для многих более сложных деревьев и алгоритмов. Не накладывает ограничений на порядок элементов.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(n)$	$O(n)$
Удаление элемента	$O(1)$	$O(n)$	$O(n)$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$
Вычисление высоты дерева	$O(1)$	$O(n)$	$O(n)$

Таблица 10: Сложность операций для бинарного дерева

2.3.2 Двоичное дерево поиска (Binary search tree — BST)

Двоичное дерево поиска — это упорядоченная версия бинарного дерева, где для каждого узла все элементы в левом поддереве меньше, а в правом поддереве больше значения самого узла. Это свойство позволяет эффективно выполнять поиск, вставку и удаление элементов.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(n)$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(n)$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(n)$
Вычисление высоты дерева	$O(1)$	$O(\log(n))$	$O(n)$

Таблица 11: Сложность операций для двоичного дерева поиска

2.3.3 АВЛ-дерево (AVL tree)

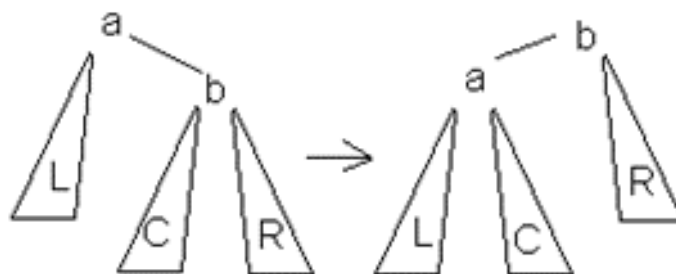
АВЛ-дерево — это сбалансированное двоичное дерево поиска, в котором для каждой вершины высота её двух поддеревьев различается не более чем на 1. Это свойство гарантирует, что дерево остаётся сбалансированным после операций вставки и удаления, обеспечивая логарифмическую сложность операций.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Удаление элемента	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Вычисление высоты дерева	$O(1)$	$O(n)$	$O(n)$

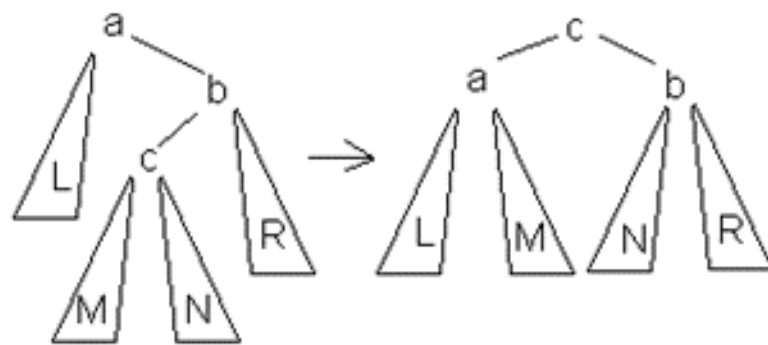
Таблица 12: Сложность операций для АВЛ-дерева

Для поддержания сбалансированности дерева используются повороты. Вот все их типы:

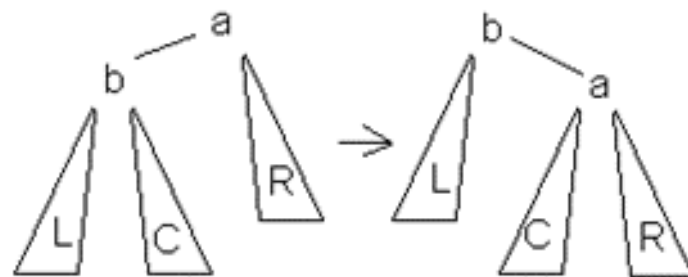
- Малое левое вращение



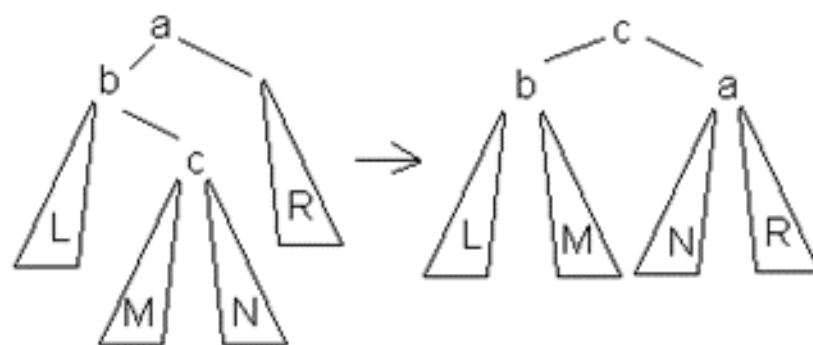
- Большое левое вращение



- Малое правое вращение



- Большое правое вращение



2.3.4 Красно-чёрное дерево (Red-black tree)

Красно-чёрное дерево — это самобалансирующееся двоичное дерево поиска, которое гарантирует логарифмическую высоту дерева при любых последовательностях операций. Балансировка достигается за счёт использования дополнительного цвета (красный/чёрный) для каждого узла и соблюдения набора свойств: 1) Каждый узел либо красный, либо чёрный. 2) Корень всегда чёрный. 3) Все листья (NIL) чёрные. 4) У красного узла оба потомка чёрные. 5) Все пути от узла до листьев содержат одинаковое количество чёрных узлов.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Вычисление высоты дерева	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$

Таблица 13: Сложность операций для красно-чёрного дерева

Для балансировки красно-чёрного дерева есть следующие операции:

- Левый поворот
- Правый поворот
- Перекрашивание узлов

2.3.5 Дерево отрезков (Segment tree)

Дерево отрезков — это древовидная структура данных, которая позволяет эффективно выполнять операции на отрезках массива (диапазонах элементов). Она предварительно вычисляет и хранит агрегированную информацию об отрезках, что позволяет быстро отвечать на запросы.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(n)$	$O(n)$	$O(n)$
Удаление элемента	$O(n)$	$O(n)$	$O(n)$
Обновление элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(n)$	$O(n)$	$O(n)$
Вычисление высоты дерева	$O(1)$	$O(1)$	$O(1)$

Таблица 14: Сложность операций для дерева отрезков

2.3.6 B-дерево (B-tree)

B-дерево — это сбалансированное дерево поиска. Каждый узел может содержать множество ключей (кроме корня) и иметь много потомков, что минимизирует высоту дерева. Также в B-дереве все листья находятся на одном уровне.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Вычисление высоты дерева	$O(1)$	$O(\log(n))$	$O(\log(n))$

Таблица 15: Сложность операций для B-дерева

2.3.7 B+-дерево (B+-tree)

B+-дерево — это вариант B-дерева, где все данные хранятся только в листьях, а внутренние узлы содержат только ключи-указатели. Это делает структуру особенно эффективной для диапазонных запросов и последовательного доступа. Также в B+-дереве все листья связаны между собой и образуют связанный список.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Диапазонный запрос с k элементами	$O(1)$	$O(k + \log(n))$	$O(k + \log(n))$
Вычисление высоты дерева	$O(1)$	$O(\log(n))$	$O(\log(n))$

Таблица 16: Сложность операций для B+-дерева

2.3.8 Куча (Heap)

Куча — это специализированная древовидная структура данных, которая удовлетворяет свойству кучи: каждый родитель больше своих потомков для максимальной кучи или каждый родитель меньше своих потомков для минимальной кучи. Это позволяет эффективно находить и извлекать элемент с наивысшим (или наименьшим) приоритетом.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(\log(n))$
Удаление элемента	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$
Поиск элемента	$O(1)$	$O(n)$	$O(n)$
Изменение приоритета	$O(1)$	$O(n)$	$O(n)$

Таблица 17: Сложность операций для кучи

2.3.9 Двоичная куча (Binary heap)

Двоичная куча — это частный случай обычной кучи, когда каждый узел имеет не более двух потомков.

2.3.10 Префиксное дерево (Trie)

Префиксное дерево — это древовидная структура данных для эффективного хранения и поиска строк. Ключи обычно являются строками, а узлы хранят символы и ссылки на следующие символы, образуя пути от корня до листьев, соответствующие строкам.

Операция	Лучший случай	Средний случай	Худший случай
Вставка слова длины m	$O(1)$	$O(m)$	$O(m)$
Удаление слова длины m	$O(1)$	$O(m)$	$O(m)$
Поиск слова длины m	$O(1)$	$O(m)$	$O(m)$
Авто дополнение k слов с префиксом длины m	$O(1)$	$O(k + m)$	$O(k + m)$
Поиск всех слов	$O(1)$	$O(n)$	$O(n)$

Таблица 18: Сложность операций для префиксного дерева

2.3.11 Декартово дерево (Treap/Cartesian tree)

Декартово дерево — это структура данных, сочетающая в себе свойства бинарного дерева поиска и кучи. Каждый узел содержит два ключа: один используется для упорядочивания как в бинарном дереве поиска, а другой — как в куче.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(n)$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(n)$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(n)$
Объединение двух деревьев	$O(1)$	$O(\log(n))$	$O(n)$
Разделение дерева	$O(1)$	$O(\log(n))$	$O(n)$

Таблица 19: Сложность операций для декартова дерева

2.3.12 Splay-дерево (Splay tree)

Splay-дерево — это самобалансирующееся бинарное дерево поиска, которое выполняет операцию «splay» (перемещение) для элемента, к которому был выполнен доступ, перемещая его в корень дерева. Это позволяет часто используемым элементам быть ближе к корню.

Операция	Лучший случай	Средний случай	Худший случай
Вставка элемента	$O(1)$	$O(\log(n))$	$O(n)$
Удаление элемента	$O(1)$	$O(\log(n))$	$O(n)$
Поиск элемента	$O(1)$	$O(\log(n))$	$O(n)$
Доступ к элементу	$O(1)$	$O(\log(n))$	$O(n)$

Таблица 20: Сложность операций для Splay-дерева

2.3.13 Дерево квадрантов (Quadtree)

Дерево квадрантов — это древовидная структура данных, в которой каждый внутренний узел имеет ровно четыре потомка. Используется для разбиения двумерного пространства путем рекурсивного деления его на четыре квадранта.

Операция	Лучший случай	Средний случай	Худший случай
Вставка точки	$O(1)$	$O(\log(n))$	$O(n)$
Удаление точки	$O(1)$	$O(\log(n))$	$O(n)$
Поиск точки	$O(1)$	$O(\log(n))$	$O(n)$
Поиск в области	$O(1)$	$O(\sqrt{n} + k)$	$O(n)$

Таблица 21: Сложность операций для дерева квадрантов

2.3.14 Дерево восьмиричных разбиений (Octree)

Дерево восьмиричных разбиений — это древовидная структура данных, в которой каждый внутренний узел имеет ровно восемь потомков. Используется для разбиения трехмерного пространства путем рекурсивного деления его на восемь октантов.

Операция	Лучший случай	Средний случай	Худший случай
Вставка точки	$O(1)$	$O(\log(n))$	$O(n)$
Удаление точки	$O(1)$	$O(\log(n))$	$O(n)$
Поиск точки	$O(1)$	$O(\log(n))$	$O(n)$
Поиск в объеме	$O(1)$	$O(\sqrt[3]{n} + k)$	$O(n)$

Таблица 22: Сложность операций для дерева восьмиричных разбиений

2.3.15 Арифметическое дерево выражения (Expression tree)

Арифметическое дерево выражения — это бинарное дерево, используемое для представления арифметических выражений. Листья дерева представляют операнды, а внутренние узлы — операторы.

Операция	Лучший случай	Средний случай	Худший случай
Построение дерева	$O(n)$	$O(n)$	$O(n)$
Вычисление выражения	$O(1)$	$O(n)$	$O(n)$
Обход дерева	$O(1)$	$O(n)$	$O(n)$
Оптимизация дерева	$O(1)$	$O(n)$	$O(n^2)$

Таблица 23: Сложность операций для арифметического дерева выражения

2.3.16 Неориентированный граф (Undirected graph)

Неориентированный граф — это граф, в котором рёбра не имеют направления. Связь между вершинами является двусторонней.

Операция	Лучший случай	Средний случай	Худший случай
Добавление вершины	$O(1)$	$O(1)$	$O(1)$
Добавление ребра	$O(1)$	$O(1)$	$O(1)$
Удаление вершины	$O(1)$	$O(deg(v))$	$O(n)$
Удаление ребра	$O(1)$	$O(1)$	$O(1)$
Поиск соседей	$O(1)$	$O(deg(v))$	$O(n)$
Проверка смежности	$O(1)$	$O(deg(v))$	$O(n)$

Таблица 24: Сложность операций для неориентированного графа

2.3.17 Ориентированный граф (Directed graph/Digraph)

Ориентированный граф — это граф, в котором рёбра имеют направление. Связь между вершинами является односторонней.

Операция	Лучший случай	Средний случай	Худший случай
Добавление вершины	$O(1)$	$O(1)$	$O(1)$
Добавление ребра	$O(1)$	$O(1)$	$O(1)$
Удаление вершины	$O(1)$	$O(deg^+(v) + deg^-(v))$	$O(n)$
Удаление ребра	$O(1)$	$O(1)$	$O(1)$
Поиск исходящих соседей	$O(1)$	$O(deg^+(v))$	$O(n)$
Поиск входящих соседей	$O(1)$	$O(deg^-(v))$	$O(n)$

Таблица 25: Сложность операций для ориентированного графа

2.3.18 Взвешенный граф (Weighted graph)

Взвешенный граф — это граф, в котором каждому ребру присвоен числовой вес (стоимость, длина и т.д.).

Операция	Лучший случай	Средний случай	Худший случай
Добавление ребра с весом	$O(1)$	$O(1)$	$O(1)$
Изменение веса ребра	$O(1)$	$O(1)$	$O(1)$
Поиск минимального пути	$O(1)$	$O(n \log n + m)$	$O(n \log n + m)$
Поиск минимального остовного дерева	$O(1)$	$O(m \log n)$	$O(m \log n)$

Таблица 26: Сложность операций для взвешенного графа

2.3.19 Граф с метками (Labeled graph)

Граф с метками — это граф, в котором вершины и/или рёбра имеют метки (названия, категории и т.д.).

Операция	Лучший случай	Средний случай	Худший случай
Поиск по метке вершины	$O(1)$	$O(n)$	$O(n)$
Поиск по метке ребра	$O(1)$	$O(m)$	$O(m)$
Изменение метки	$O(1)$	$O(1)$	$O(1)$
Группировка по меткам	$O(1)$	$O(n + m)$	$O(n + m)$

Таблица 27: Сложность операций для графа с метками

2.3.20 Лес (Forest)

Лес — это множество непересекающихся деревьев, или ациклический неориентированный граф.

Операция	Лучший случай	Средний случай	Худший случай
Добавление дерева	$O(1)$	$O(1)$	$O(1)$
Объединение деревьев	$O(1)$	$O(n)$	$O(n)$
Поиск компоненты связности	$O(1)$	$O(\alpha(n))$	$O(\alpha(n))$
Подсчет деревьев	$O(1)$	$O(1)$	$O(1)$

Таблица 28: Сложность операций для леса

2.3.21 Связный граф (Connected graph)

Связный граф — это граф, в котором существует путь между любой парой вершин.

Операция	Лучший случай	Средний случай	Худший случай
Проверка связности	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск компонент связности	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск мостов	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск точек сочленения	$O(1)$	$O(n + m)$	$O(n + m)$

Таблица 29: Сложность операций для связного графа

2.3.22 Полный граф (Complete graph)

Полный граф — это граф, в котором каждая пара различных вершин соединена ребром.

Операция	Лучший случай	Средний случай	Худший случай
Добавление вершины	$O(n)$	$O(n)$	$O(n)$
Удаление вершины	$O(n)$	$O(n)$	$O(n)$
Проверка смежности	$O(1)$	$O(1)$	$O(1)$
Подсчет ребер	$O(1)$	$O(1)$	$O(1)$

Таблица 30: Сложность операций для полного графа

2.3.23 Двудольный граф (Bipartite graph)

Двудольный граф — это граф, вершины которого можно разбить на два непересекающихся множества так, что каждое ребро соединяет вершины из разных множеств.

Операция	Лучший случай	Средний случай	Худший случай
Проверка двудольности	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск максимального паросочетания	$O(1)$	$O(m\sqrt{n})$	$O(m\sqrt{n})$
Разбиение на доли	$O(1)$	$O(n + m)$	$O(n + m)$
Проверка полной двудольности	$O(1)$	$O(n + m)$	$O(n + m)$

Таблица 31: Сложность операций для двудольного графа

2.3.24 Ациклический ориентированный граф (Directed acyclic graph — DAG)

Ациклический ориентированный граф — это ориентированный граф без направленных циклов. Широко используется для представления структур с зависимостями.

Операция	Лучший случай	Средний случай	Худший случай
Топологическая сортировка	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск критического пути	$O(1)$	$O(n + m)$	$O(n + m)$
Проверка достижимости	$O(1)$	$O(n + m)$	$O(n + m)$
Поиск транзитивного замыкания	$O(1)$	$O(n^2)$	$O(n^2)$

Таблица 32: Сложность операций для ациклического ориентированного графа

2.3.25 Матрица (Matrix)

Матрица — это двумерная структура данных, состоящая из строк и столбцов. Каждый элемент матрицы имеет два индекса: номер строки и номер столбца.

Операция	Лучший случай	Средний случай	Худший случай
Доступ к элементу	$O(1)$	$O(1)$	$O(1)$
Изменение элемента	$O(1)$	$O(1)$	$O(1)$
Транспонирование	$O(1)$	$O(n \cdot m)$	$O(n \cdot m)$
Сложение матриц	$O(1)$	$O(n \cdot m)$	$O(n \cdot m)$
Умножение матриц	$O(1)$	$O(n \cdot m \cdot k)$	$O(n \cdot m \cdot k)$

Таблица 33: Сложность операций для матрицы

2.3.26 Тензор (Tensor)

Тензор — это многомерная структура данных, обобщающая понятия скаляра (0-мерный тензор), вектора (1-мерный тензор) и матрицы (2-мерный тензор). Тензоры могут иметь произвольное количество измерений.

Операция	Лучший случай	Средний случай	Худший случай
Доступ к элементу	$O(1)$	$O(1)$	$O(1)$
Изменение элемента	$O(1)$	$O(1)$	$O(1)$
Сложение тензоров	$O(1)$	$O(\prod d_i)$	$O(\prod d_i)$
Тензорное произведение	$O(1)$	$O(\prod d_i \cdot \prod e_j)$	$O(\prod d_i \cdot \prod e_j)$
Свертка	$O(1)$	$O(\prod d_i)$	$O(\prod d_i)$

Таблица 34: Сложность операций для тензора

3 Алгоритмы

Алгоритм — это конечная последовательность четко определенных инструкций, предназначенная для решения конкретной задачи или класса задач. Алгоритмы являются фундаментальным понятием в информатике и лежат в основе всех компьютерных программ.

Ключевые характеристики алгоритмов:

- Конечность: алгоритм должен завершаться после выполнения конечного числа шагов.
- Определенность: каждая инструкция должна быть четкой и однозначной.
- Входные данные: алгоритм принимает ноль или более входных значений.
- Выходные данные: алгоритм производит один или более результатов.
- Эффективность: алгоритм должен использовать разумное количество ресурсов (время, память).

3.1 Алгоритмы сортировки

Алгоритмы сортировки предназначены для упорядочивания элементов в структуре данных согласно определенному критерию (обычно по возрастанию или убыванию). Выбор алгоритма сортировки зависит от размера данных, их частичной упорядоченности и требований к стабильности.

3.1.1 Сортировка пузырьком (Bubble sort)

Сортировка пузырьком — это простой алгоритм сортировки, который многократно проходит по массиву, сравнивает соседние элементы и меняет их местами, если они находятся в неправильном порядке. Проходы повторяются до тех пор, пока массив не будет отсортирован.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n^2)$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Да	Да	Да
Адаптивность	Да	Нет	Нет

Таблица 35: Характеристики сортировки пузырьком

3.1.2 Сортировка выбором (Selection sort)

Сортировка выбором — это алгоритм, который на каждом проходе находит минимальный (или максимальный) элемент в неотсортированной части массива и помещает его в конец отсортированной части.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n^2)$	$O(n^2)$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Нет	Нет	Нет
Адаптивность	Нет	Нет	Нет

Таблица 36: Характеристики сортировки выбором

3.1.3 Сортировка вставками (Insertion sort)

Сортировка вставками — это алгоритм, который строит отсортированную последовательность постепенно, вставляя каждый следующий элемент в правильную позицию среди уже отсортированных элементов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n^2)$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Да	Да	Да
Адаптивность	Да	Нет	Нет

Таблица 37: Характеристики сортировки вставками

3.1.4 Сортировка слиянием (Merge sort)

Сортировка слиянием — это алгоритм «разделяй и властвуй», который рекурсивно разбивает массив на две половины, сортирует их, а затем объединяет отсортированные половины в один отсортированный массив.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Стабильность	Да	Да	Да
Адаптивность	Нет	Нет	Нет

Таблица 38: Характеристики сортировки слиянием

3.1.5 Быстрая сортировка (Quick sort)

Быстрая сортировка — это алгоритм «разделяй и властвуй», который выбирает опорный элемент, разбивает массив на две части (элементы меньше опорного и элементы больше опорного), а затем рекурсивно сортирует эти части.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Пространственная сложность	$O(\log n)$	$O(\log n)$	$O(n)$
Стабильность	Нет	Нет	Нет
Адаптивность	Нет	Нет	Нет

Таблица 39: Характеристики быстрой сортировки

3.1.6 Пирамидальная сортировка (Heap sort)

Пирамидальная сортировка — это алгоритм, который использует структуру данных «куча» для сортировки элементов. Алгоритм строит max-кучу из массива, затем многократно извлекает максимальный элемент и перестраивает кучу.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Нет	Нет	Нет
Адаптивность	Нет	Нет	Нет

Таблица 40: Характеристики пирамидальной сортировки

3.1.7 Сортировка Шелла (Shell sort)

Сортировка Шелла — это алгоритм, который является обобщением сортировки вставками. Он сортирует элементы, находящиеся на определенном расстоянии друг от друга, постепенно уменьшая это расстояние.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n^{1.5})$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Нет	Нет	Нет
Адаптивность	Да	Нет	Нет

Таблица 41: Характеристики сортировки Шелла

3.1.8 Сортировка деревом (Tree sort)

Сортировка деревом — это алгоритм, который использует бинарное дерево поиска для сортировки элементов. Элементы вставляются в дерево, а затем обходом в порядке возрастания извлекаются в отсортированном виде.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Стабильность	Да	Да	Да
Адаптивность	Нет	Нет	Нет

Таблица 42: Характеристики сортировки деревом

3.1.9 Сортировка подсчётом (Counting sort)

Сортировка подсчётом — это алгоритм, который работает путем подсчета количества элементов с каждым distinct значением, а затем использует эту информацию для определения позиций элементов в отсортированной последовательности. Эффективен, когда диапазон значений ограничен.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n + k)$	$O(n + k)$	$O(n + k)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Стабильность	Да	Да	Да
Адаптивность	Нет	Нет	Нет

Таблица 43: Характеристики сортировки подсчётом

3.1.10 Поразрядная сортировка (Radix sort)

Поразрядная сортировка — это алгоритм, который сортирует числа поразрядно, начиная с младшего разряда и заканчивая старшим. Для сортировки каждого разряда обычно используется стабильный алгоритм сортировки (часто сортировка подсчётом).

Случай	Лучший	Средний	Худший
Временная сложность	$O(nk)$	$O(nk)$	$O(nk)$
Пространственная сложность	$O(n + k)$	$O(n + k)$	$O(n + k)$
Стабильность	Да	Да	Да
Адаптивность	Нет	Нет	Нет

Таблица 44: Характеристики поразрядной сортировки

3.1.11 Блочная/Карманная сортировка (Bucket sort)

Блочная сортировка — это алгоритм, который распределяет элементы по «ведрам» (блокам), затем сортирует каждое ведро отдельно и объединяет результаты.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n + k)$	$O(n + k)$	$O(n^2)$
Пространственная сложность	$O(n + k)$	$O(n + k)$	$O(n + k)$
Стабильность	Да	Да	Да
Адаптивность	Нет	Нет	Нет

Таблица 45: Характеристики блочной сортировки

3.1.12 Тимо-сортировка (Tim sort)

Тимо-сортировка — это гибридный алгоритм, сочетающий сортировку вставками и слиянием. Оптимизирован для работы с реальными данными, часто содержащими упорядоченные подпоследовательности.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Стабильность	Да	Да	Да
Адаптивность	Да	Да	Да

Таблица 46: Характеристики Тимо-сортировки

3.1.13 Интроспективная сортировка (Introspective sort)

Интроспективная сортировка — это гибридный алгоритм, который начинает как быстрая сортировка, но переключается на пирамидальную сортировку, если глубина рекурсии становится слишком большой.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(\log n)$	$O(\log n)$	$O(\log n)$
Стабильность	Нет	Нет	Нет
Адаптивность	Нет	Нет	Нет

Таблица 47: Характеристики интроспективной сортировки

3.1.14 Гномья сортировка (Gnome sort)

Гномья сортировка — это простой алгоритм, похожий на сортировку вставками, но работающий путем попарного сравнения и обмена соседних элементов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n^2)$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Да	Да	Да
Адаптивность	Да	Нет	Нет

Таблица 48: Характеристики гномьей сортировки

3.1.15 Сортировка расчёской (Comb sort)

Сортировка расчёской — это улучшение сортировки пузырьком, которое использует увеличивающийся промежуток между сравниваемыми элементами.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n^2)$	$O(n^2)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Нет	Нет	Нет
Адаптивность	Да	Нет	Нет

Таблица 49: Характеристики сортировки расчёской

3.1.16 Глупая сортировка (Bogo sort)

Глупая сортировка — это неэффективный алгоритм, который случайным образом переставляет элементы до тех пор, пока массив не окажется отсортированным.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n \cdot n!)$	Бесконечность
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Стабильность	Нет	Нет	Нет
Адаптивность	Нет	Нет	Нет

Таблица 50: Характеристики глупой сортировки

3.2 Алгоритмы поиска

Алгоритмы поиска предназначены для нахождения элемента в структуре данных. Эффективность поиска сильно зависит от организации данных и выбранного алгоритма.

3.2.1 Линейный поиск (Linear search)

Линейный поиск — это простейший алгоритм поиска, который последовательно проверяет каждый элемент структуры данных до тех пор, пока не найдет искомый элемент или не проверит все элементы.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 51: Характеристики линейного поиска

3.2.2 Двоичный поиск (Binary search)

Двоичный поиск — это эффективный алгоритм поиска в отсортированном массиве. На каждом шаге алгоритм сравнивает искомый элемент с элементом в середине массива и, в зависимости от результата сравнения, отбрасывает одну из половин массива.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\log n)$	$O(\log n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 52: Характеристики двоичного поиска

3.2.3 Поиск прыжками (Jump search)

Поиск прыжками — это алгоритм для отсортированных массивов, который проверяет меньшее количество элементов по сравнению с линейным поиском, перескакивая через фиксированное количество элементов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 53: Характеристики поиска прыжками

3.2.4 Интерполяционный поиск (Interpolation search)

Интерполяционный поиск — это улучшение двоичного поиска для равномерно распределенных данных. Алгоритм оценивает позицию искомого элемента на основе значений границ текущего интервала.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\log \log n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 54: Характеристики интерполяционного поиска

3.2.5 Экспоненциальный поиск (Exponential search)

Экспоненциальный поиск — это алгоритм для поиска в неограниченных или бесконечных отсортированных массивах. Сначала находится диапазон, в котором может находиться элемент, затем применяется двоичный поиск.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\log n)$	$O(\log n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 55: Характеристики экспоненциального поиска

3.2.6 Поиск хеш-таблицой (Hash table search)

Поиск в хеш-таблице использует хеш-функцию для вычисления индекса элемента. При правильной реализации обеспечивает постоянное время доступа в среднем случае.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(1)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$

Таблица 56: Характеристики поиска хеш-таблицой

3.2.7 Поиск в бинарном дереве (Binary tree search)

Поиск в бинарном дереве использует свойства BST: для каждого узла все элементы в левом поддереве меньше, а в правом — больше значения узла.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\log n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(\log n)$	$O(n)$

Таблица 57: Характеристики поиска бинарным деревом

3.2.8 Поиск делением на три части (Ternary search)

Тернарный поиск — это алгоритм для нахождения минимума или максимума унимодальной функции. Разделяет область поиска на три части и отбрасывает одну из них.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(\log_3 n)$	$O(\log_3 n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 58: Характеристики поиска делением на три части

3.3 Алгоритмы на графах

Алгоритмы на графах решают задачи, связанные с обходом, поиском путей, определением связности и другими операциями на графовых структурах.

3.3.1 Поиск в ширину (Breadth-first search — BFS)

Поиск в ширину — это алгоритм обхода или поиска в графах и деревьях, который исследует все соседние вершины на текущей глубине перед переходом к вершинам на следующем уровне.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 59: Характеристики поиска в ширину

3.3.2 Поиск в глубину (Depth-first search — DFS)

Поиск в глубину — это алгоритм обхода или поиска в графах и деревьях, который исследует ветви настолько глубоко, насколько это возможно, прежде чем возвращаться назад.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 60: Характеристики поиска в глубину

3.3.3 Алгоритм Дейкстры (Dijkstra algorithm)

Алгоритм Дейкстры находит кратчайшие пути от одной вершины (источника) до всех остальных вершин в взвешенном графе с неотрицательными весами рёбер.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E + V \log V)$	$O(E + V \log V)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 61: Характеристики алгоритма Дейкстры

3.3.4 Алгоритм Беллмана-Форда (Bellman-Ford algorithm)

Алгоритм Беллмана-Форда находит кратчайшие пути от одной вершины до всех остальных в взвешенном графе, который может содержать рёбра с отрицательными весами.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(VE)$	$O(VE)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 62: Характеристики алгоритма Беллмана-Форда

3.3.5 Алгоритм Флойда-Уоршелла (Floyd-Warshall algorithm)

Алгоритм Флойда-Уоршелла находит кратчайшие пути между всеми парами вершин в взвешенном графе. Может работать с отрицательными весами, но не с отрицательными циклами.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V^3)$	$O(V^3)$
Пространственная сложность	$O(1)$	$O(V^2)$	$O(V^2)$

Таблица 63: Характеристики алгоритма Флойда-Уоршелла

3.3.6 Алгоритм Прима (Prim algorithm)

Алгоритм Прима находит минимальное остовное дерево в связном взвешенном неориентированном графе. Строит дерево, начиная с произвольной вершины и последовательно добавляя к нему рёбра минимального веса.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E \log V)$	$O(E \log V)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 64: Характеристики алгоритма Прима

3.3.7 Алгоритм Краскала (Kruskal algorithm)

Алгоритм Краскала находит минимальное остовное дерево в связном взвешенном неориентированном графе. Строит дерево, последовательно добавляя рёбра в порядке возрастания их весов, избегая образования циклов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E \log V)$	$O(E \log V)$
Пространственная сложность	$O(1)$	$O(E)$	$O(E)$

Таблица 65: Характеристики алгоритма Краскала

3.3.8 Алгоритм Форда-Фалкерсона (Ford-Fulkerson algorithm)

Алгоритм Форда-Фалкерсона находит максимальный поток в транспортной сети. Работает путем поиска увеличивающих путей и увеличения потока вдоль них.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E \cdot \text{maxflow})$	$O(E \cdot \text{maxflow})$
Пространственная сложность	$O(1)$	$O(V + E)$	$O(V + E)$

Таблица 66: Характеристики алгоритма Форда-Фалкерсона

3.3.9 Алгоритм Эдмондса-Карпа (Edmonds-Karp algorithm)

Алгоритм Эдмондса-Карпа — это реализация алгоритма Форда-Фалкерсона, использующая поиск в ширину для нахождения увеличивающих путей.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(VE^2)$	$O(VE^2)$
Пространственная сложность	$O(1)$	$O(V + E)$	$O(V + E)$

Таблица 67: Характеристики алгоритма Эдмондса-Карпа

3.3.10 Алгоритм Диница (Dinic algorithm)

Алгоритм Диница находит максимальный поток в сети. Использует слоистые сети и блокирующие потоки для более эффективного вычисления.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V^2E)$	$O(V^2E)$
Пространственная сложность	$O(1)$	$O(V + E)$	$O(V + E)$

Таблица 68: Характеристики алгоритма Диница

3.3.11 Поиск компонент связности (Connected components)

Поиск компонент связности находит все связные подграфы в неориентированном графе. Обычно реализуется с помощью поиска в глубину или ширину.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 69: Характеристики поиска компонент связности

3.3.12 Поиск мостов и точек сочленения (Bridges & Articulation points)

Поиск мостов (рёбер, удаление которых увеличивает количество компонент связности) и точек сочленения (вершин, удаление которых увеличивает количество компонент связности).

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 70: Характеристики поиска мостов и точек сочленения

3.3.13 Алгоритм Косарайю (Kosaraju algorithm)

Алгоритм Косарайю находит сильно связные компоненты в ориентированном графе. Использует два обхода в глубину: на исходном графе и на транспонированном.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 71: Характеристики алгоритма Косарайю

3.3.14 Алгоритм Тарьяна (Tarjan algorithm)

Алгоритм Тарьяна находит сильно связные компоненты в ориентированном графе за один обход в глубину, используя стек и метки вершин.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 72: Характеристики алгоритма Тарьяна

3.3.15 Топологическая сортировка (Topological sort)

Топологическая сортировка — это упорядочивание вершин ориентированного ациклического графа (DAG) таким образом, что для любого направленного ребра (u, v) вершина u comes before v в упорядочении.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V + E)$	$O(V + E)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 73: Характеристики топологической сортировки

3.3.16 Поиск эйлерова пути (Eulerian path)

Поиск эйлерова пути находит путь в графе, который проходит через каждое ребро ровно один раз. Эйлеров цикл — это замкнутый эйлеров путь.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E)$	$O(E)$
Пространственная сложность	$O(1)$	$O(E)$	$O(E)$

Таблица 74: Характеристики поиска эйлерова пути

3.3.17 Поиск гамильтонова пути (Hamiltonian path)

Поиск гамильтонова пути находит путь в графе, который проходит через каждую вершину ровно один раз. Гамильтонов цикл — это замкнутый гамильтонов путь.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(V!)$	$O(V!)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 75: Характеристики поиска гамильтонова пути

3.3.18 Раскраска графов (Graph coloring)

Раскраска графов — это присвоение цветов вершинам графа таким образом, что никакие две смежные вершины не имеют одинаковый цвет. Задача нахождения минимального количества цветов является NP-трудной.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(2^V)$	$O(2^V)$
Пространственная сложность	$O(1)$	$O(V)$	$O(V)$

Таблица 76: Характеристики раскраски графов

3.3.19 Паросочетания в графах (Graph matching)

Паросочетание — это набор рёбер, не имеющих общих вершин. Задача о максимальном паросочетании находит паросочетание с максимальным количеством рёбер.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(E\sqrt{V})$	$O(E\sqrt{V})$
Пространственная сложность	$O(1)$	$O(V + E)$	$O(V + E)$

Таблица 77: Характеристики паросочетаний в графах

3.3.20 Поиск A* (A* search)

A* — это алгоритм поиска пути, который использует эвристическую функцию для оценки стоимости пути до цели. Сочетает преимущества поиска в ширину и жадного поиска.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(b^d)$	$O(b^d)$
Пространственная сложность	$O(1)$	$O(b^d)$	$O(b^d)$

Таблица 78: Характеристики поиска A*

3.3.21 Поиск по первому лучшему (Best-first search)

Поиск по первому лучшему — это алгоритм, который расширяет наиболее перспективный узел согласно эвристической функции, без учета стоимости достижения этого узла.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(b^d)$	$O(b^d)$
Пространственная сложность	$O(1)$	$O(b^d)$	$O(b^d)$

Таблица 79: Характеристики поиска по первому лучшему

3.3.22 Жадный поиск (Greedy search)

Жадный поиск — это алгоритм, который на каждом шаге выбирает локально оптимальное решение, надеясь найти глобальный оптимум.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(b^m)$	$O(b^m)$
Пространственная сложность	$O(1)$	$O(b^m)$	$O(b^m)$

Таблица 80: Характеристики жадного поиска

3.4 Алгоритмы на строках

Алгоритмы на строках решают задачи поиска подстрок, сравнения строк, вычисления расстояний между строками и другие задачи обработки текстовых данных.

3.4.1 Наивный алгоритм (Naive algorithm)

Наивный алгоритм поиска подстроки проверяет все возможные позиции в тексте, сравнивая шаблон с каждой подстрокой текста.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(nm)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 81: Характеристики наивного алгоритма

3.4.2 Алгоритм Кнута-Морриса-Практа (Knuth-Morris-Pratt algorithm)

Алгоритм КМП использует префикс-функцию для избежания повторных сравнений при поиске подстроки. Предварительно обрабатывает шаблон для создания таблицы сдвигов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n + m)$	$O(n + m)$
Пространственная сложность	$O(m)$	$O(m)$	$O(m)$

Таблица 82: Характеристики алгоритма Кнута-Морриса-Практа

3.4.3 Алгоритм Рабина-Карпа (Rabin-Karp algorithm)

Алгоритм Рабина-Карпа использует хеширование для поиска подстроки. Вычисляет хеш-значение для шаблона и для всех подстрок текста, сравнивая их.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n + m)$	$O(n + m)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 83: Характеристики алгоритма Рабина-Карпа

3.4.4 Алгоритм Бойера-Мура (Boyer-Moore algorithm)

Алгоритм Бойера-Мура использует две эвристики: таблицу плохого символа и таблицу хорошего суффикса для пропуска заведомо неподходящих позиций.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n/m)$	$O(n + m)$	$O(nm)$
Пространственная сложность	$O(m)$	$O(m)$	$O(m)$

Таблица 84: Характеристики алгоритма Бойера-Мура

3.4.5 Алгоритм Ахо-Корасик (Aho-Corasick algorithm)

Алгоритм Ахо-Корасик находит все вхождения множества шаблонов в тексте. Строит конечный автомат по набору шаблонов для эффективного поиска.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n + m + z)$	$O(n + m + z)$	$O(n + m + z)$
Пространственная сложность	$O(m)$	$O(m)$	$O(m)$

Таблица 85: Характеристики алгоритма Ахо-Корасик

3.4.6 Расстояние Левенштейна (Levenshtein distance)

Расстояние Левенштейна — это минимальное количество операций вставки, удаления и замены символов, необходимых для преобразования одной строки в другую.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(nm)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(\min(n, m))$	$O(\min(n, m))$

Таблица 86: Характеристики расстояния Левенштейна

3.4.7 Расстояние Хэмминга (Hamming distance)

Расстояние Хэмминга — это количество позиций, в которых соответствующие символы двух строк одинаковой длины различаются.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$

Таблица 87: Характеристики расстояния Хэмминга

3.4.8 Наибольшая общая подпоследовательность (Longest common subsequence — LCS)

Наибольшая общая подпоследовательность — это самая длинная последовательность символов, которая является подпоследовательностью обеих строк.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(nm)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(\min(n, m))$	$O(\min(n, m))$

Таблица 88: Характеристики поиска наибольшей общей подпоследовательности

3.4.9 Наибольшая общая подстрока (Longest common substring)

Наибольшая общая подстрока — это самая длинная непрерывная последовательность символов, которая является подстрокой обеих строк.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(nm)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(\min(n, m))$	$O(\min(n, m))$

Таблица 89: Характеристики поиска наибольшей общей подстроки

3.4.10 Алгоритм Вагнера-Фишера (Wagner-Fisher algorithm)

Алгоритм Вагнера-Фишера — это динамическое программирование для вычисления расстояния Левенштейна между двумя строками.

Случай	Лучший	Средний	Худший
Временная сложность	$O(1)$	$O(nm)$	$O(nm)$
Пространственная сложность	$O(1)$	$O(\min(n, m))$	$O(\min(n, m))$

Таблица 90: Характеристики алгоритма Вагнера-Фишера

4 Алгоритмы преобразования данных

Алгоритмы преобразования данных предназначены для изменения формата, структуры или представления данных с целью оптимизации хранения, передачи или обработки. К ним относятся алгоритмы сжатия, шифрования и численных преобразований.

4.1 Алгоритмы сжатия данных

Алгоритмы сжатия данных предназначены для уменьшения объема данных при сохранении содержащейся в них информации (сжатие без потерь) или с допустимой потерей части информации (сжатие с потерями).

4.1.1 Алгоритм Хаффмана (Huffman coding)

Алгоритм Хаффмана — это алгоритм сжатия данных без потерь, который использует переменные по длине кодовые слова для кодирования символов. Часто встречающиеся символы кодируются более короткими кодами, редко встречающиеся — более длинными.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 91: Характеристики алгоритма Хаффмана

4.1.2 Алгоритм Лемпеля-Зива (LZ77, LZ78, LZW)

Алгоритмы семейства LZ используют словарный подход к сжатию, заменяя повторяющиеся последовательности символов ссылками на предыдущие вхождения.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 92: Характеристики алгоритмов Лемпеля-Зива

4.1.3 Арифметическое кодирование (Arithmetic coding)

Арифметическое кодирование представляет всю входную последовательность как одно дробное число в интервале $[0, 1)$, обеспечивая более высокую степень сжатия чем Хаффман.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 93: Характеристики арифметического кодирования

4.1.4 Кодирование длин серий (Run-length encoding - RLE)

RLE заменяет последовательности одинаковых символов на пары (символ, количество повторений). Эффективно для данных с длинными сериями.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 94: Характеристики кодирования длин серий

4.1.5 BWT-преобразование (Burrows-Wheeler transform)

BWT переставляет символы строки таким образом, что одинаковые символы группируются вместе, что улучшает сжатие другими методами.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 95: Характеристики BWT-преобразования

4.1.6 Deflate

Deflate — это популярный алгоритм сжатия, сочетающий LZ77 и кодирование Хаффмана. Используется в ZIP, gzip и PNG.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 96: Характеристики алгоритма Deflate

4.1.7 Brotli

Brotli — это современный алгоритм сжатия от Google, обеспечивающий лучшее сжатие чем Deflate при сравнимой скорости.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 97: Характеристики алгоритма Brotli

4.1.8 LZ4

LZ4 — это алгоритм сжатия, ориентированный на высокую скорость распаковки. Широко используется в реальном времени.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 98: Характеристики алгоритма LZ4

4.1.9 Zstandard

Zstandard — это современный алгоритм сжатия от Facebook, обеспечивающий хороший баланс между степенью сжатия и скоростью.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(n)$	$O(n)$	$O(n)$
Тип сжатия	Без потерь		

Таблица 99: Характеристики алгоритма Zstandard

4.2 Криптографические алгоритмы

Криптографические алгоритмы обеспечивают конфиденциальность, целостность и аутентификацию данных. Они используются для шифрования, дешифрования, создания цифровых подписей и хеширования.

4.2.1 AES (Advanced Encryption Standard)

AES — это симметричный блочный шифр, принятый в качестве стандарта шифрования правительством США. Использует фиксированный размер блока 128 бит и ключи длиной 128, 192 или 256 бит.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Тип шифрования	Симметричный		

Таблица 100: Характеристики алгоритма AES

4.2.2 DES/3DES (Data Encryption Standard)

DES — это устаревший симметричный блочный шифр с ключом 56 бит. 3DES применяет DES три раза с разными ключами для увеличения безопасности.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Тип шифрования	Симметричный		

Таблица 101: Характеристики алгоритмов DES/3DES

4.2.3 Blowfish/Twofish

Blowfish и Twofish — это симметричные блочные шифры, разработанные Брюсом Шнайером. Twofish был финалистом конкурса AES.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Тип шифрования	Симметричный		

Таблица 102: Характеристики алгоритмов Blowfish/Twofish

4.2.4 ChaCha20

ChaCha20 — это потоковый шифр, разработанный как альтернатива RC4. Обеспечивает высокую скорость и безопасность.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Тип шифрования	Симметричный		

Таблица 103: Характеристики алгоритма ChaCha20

4.2.5 RC4

RC4 — это потоковый шифр, широко использовавшийся в прошлом, но сейчас считающийся небезопасным из-за уязвимостей.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Тип шифрования	Симметричный		

Таблица 104: Характеристики алгоритма RC4

4.2.6 RSA (Rivest-Shamir-Adleman)

RSA — это асимметричный криптографический алгоритм, использующий пару ключей: открытый ключ для шифрования и закрытый ключ для дешифрования. Безопасность RSA основана на сложности факторизации больших чисел.

Случай	Лучший	Средний	Худший
Временная сложность	$O(k^3)$	$O(k^3)$	$O(k^3)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Тип шифрования	Асимметричный		

Таблица 105: Характеристики алгоритма RSA

4.2.7 ECC (Elliptic Curve Cryptography)

ECC — это асимметричная криптография, основанная на алгебраической структуре эллиптических кривых. Обеспечивает сравнимую безопасность с RSA при меньшей длине ключа.

Случай	Лучший	Средний	Худший
Временная сложность	$O(k^2)$	$O(k^2)$	$O(k^2)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Тип шифрования	Асимметричный		

Таблица 106: Характеристики алгоритма ECC

4.2.8 ElGamal

ElGamal — это асимметричный алгоритм шифрования, основанный на проблеме дискретного логарифмирования. Также используется для цифровых подписей.

Случай	Лучший	Средний	Худший
Временная сложность	$O(k^3)$	$O(k^3)$	$O(k^3)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Тип шифрования	Асимметричный		

Таблица 107: Характеристики алгоритма ElGamal

4.2.9 DSA (Digital Signature Algorithm)

DSA — это стандарт для цифровых подписей, основанный на проблеме дискретного логарифмирования. Используется для аутентификации сообщений.

Случай	Лучший	Средний	Худший
Временная сложность	$O(k^3)$	$O(k^3)$	$O(k^3)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Назначение	Цифровые подписи		

Таблица 108: Характеристики алгоритма DSA

4.2.10 SHA-256/SHA-3 (Secure Hash Algorithm)

SHA-256 и SHA-3 — это криптографические хеш-функции, которые производят 256-битный хеш-код. Используются для обеспечения целостности данных.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Назначение	Хеширование		

Таблица 109: Характеристики алгоритмов SHA-256/SHA-3

4.2.11 MD5

MD5 — это устаревшая хеш-функция, производящая 128-битный хеш. Считается криптографически небезопасной из-за уязвимостей к коллизиям.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Назначение	Хеширование		

Таблица 110: Характеристики алгоритма MD5

4.2.12 Bcrypt/Argon2

Bcrypt и Argon2 — это алгоритмы для хеширования паролей, специально разработанные для атак перебором. Используют соль и требуют значительных вычислительных ресурсов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(1)$	$O(1)$	$O(1)$
Назначение	Хеширование паролей		

Таблица 111: Характеристики алгоритмов Bcrypt/Argon2

4.2.13 Diffie-Hellman

Diffie-Hellman — это алгоритм обмена ключами, позволяющий двум сторонам получить общий секретный ключ через незащищенный канал связи.

Случай	Лучший	Средний	Худший
Временная сложность	$O(k^3)$	$O(k^3)$	$O(k^3)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Назначение	Обмен ключами		

Таблица 112: Характеристики алгоритма Diffie-Hellman

4.2.14 PKI (Public Key Infrastructure)

PKI — это инфраструктура открытых ключей, обеспечивающая создание, распределение и управление цифровыми сертификатами.

Случай	Лучший	Средний	Худший
Временная сложность	Зависит от реализации	Зависит от реализации	Зависит от реализации
Пространственная сложность	Зависит от реализации	Зависит от реализации	Зависит от реализации
Назначение	Управление ключами		

Таблица 113: Характеристики PKI

4.2.15 Цифровые подписи (Digital signatures)

Цифровые подписи — это криптографический механизм для проверки подлинности и целостности цифровых сообщений или документов.

Случай	Лучший	Средний	Худший
Временная сложность	$O(n)$	$O(n)$	$O(n)$
Пространственная сложность	$O(k)$	$O(k)$	$O(k)$
Назначение	Аутентификация		

Таблица 114: Характеристики цифровых подписей